DESENVOLVIMENTO ASSISTIDO POR IA · BLOCO 3

Testes e documentação assistidos

3

Sábado, 17/10/2026 · 14h às 17h

Ferramenta: Claude Code · modo manual por padrão

Pós-Graduação em Engenharia de Software e IA · CESAR School

CESAR

ABERTURA

Vocabulário do bloco

oráculo (de teste)	o que decide se o resultado está certo
teste tautológico	não pode falhar por erro do código
mock	dublê que simula uma dependência
cobertura	o que foi executado pelos testes
mutação (mutant)	falha pequena injetada no código

escore de mutação	fração de mutantes que a suíte detecta
teste de propriedade	verifica uma regra para muitas entradas
teste diferencial	compara duas implementações
ADR	registro de decisão de arquitetura
contrato	comportamento prometido a quem usa

```
assertTrue(resultado is not None)
```

Que erro real este teste pegaria?

Pergunta que usaremos o bloco todo

ABERTURA

Agenda do bloco

Hoje · 17/10

14h00
Exposição: testes, mutação, documentação

15h00
Pausa

15h10 às 16h30
AV1.3 · cobrir o código do bloco 2 e caçar o teste que só repete a implementação

16h30
Apresentação e fechamento

Disciplina · 6 blocos

B1 · 16/10
Modos e ambiente

B2 · 17/10
Escopo e refatoração

B3 · agora
Testes e docs

B4 · 23/10
Legado

B5 · 24/10
Revisão e segurança

B6 · 24/10
Produtividade e time

TESTE COMO ORÁCULO

Teste é rede de segurança e também oráculo do agente

PARA O HUMANO

Rede de segurança

Detecta regressão.

PARA O AGENTE

Oráculo

Feedback para iterar.

Sem testes, ele "termina" porque acha que terminou.

TESTE COMO ORÁCULO

Três riscos do agente com testes

RISCO Edita o teste para passar

RISCO Pula ou apaga o teste que falha

RISCO Mocka o problema para fora

TESTE COMO ORÁCULO

TDD com agente: você controla o oráculo



TRAVA "Sem poder alterar" no prompt é intenção. Para garantir, uma regra de negar edição dos testes em `/permissions`.

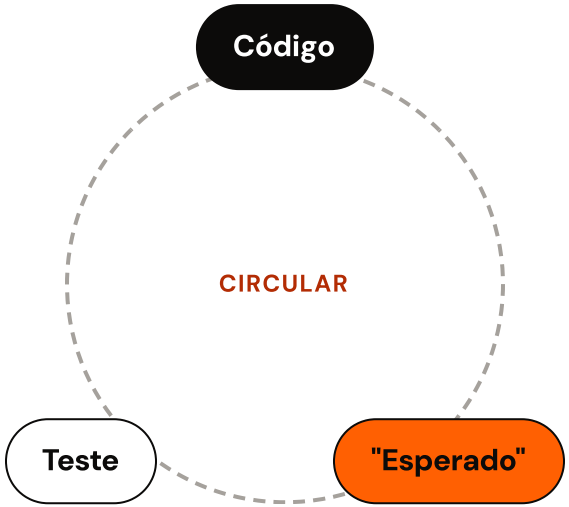
Como pedir teste de comportamento

1. Lista de casos primeiro
2. Especificação como fonte
3. Caixa-preta
4. Nome que descreve comportamento
5. Equivalência, limites, propriedades
6. Casos que podem falhar
7. Dados realistas sem dados reais

TESTE COMO ORÁCULO

O problema do oráculo

Se a IA lê só a implementação, deriva o "esperado" do próprio código: é circular.



TESTE TAUTOLÓGICO

Teste tautológico: não pode falhar por erro do código

A asserção depende do próprio código ou de um dublê.

TESTE TAUTOLÓGICO

Seis formas de teste tautológico

Forma	Como se manifesta
Espelho	Recalcula a fórmula no próprio teste
Afirma o mock	Verifica o que o dublê devolveu
Sem asserção real	Executa o código e não verifica nada
Snapshot do próprio código	Grava a saída atual como esperado, inclusive os erros
Super-mockado	Dublês até no que deveria ser testado
Passa com o corpo esvaziado	Continua verde se a função não fizer nada

Teste espelho: tautológico

A asserção reimplementa a fórmula; se a fórmula está errada, o teste também está.

TAUTOLÓGICO

```
def aplicar_desconto(total: Decimal, cupom: Cupom | None) → Decimal:
    if cupom is None:
        return total
    return total - (total * cupom.percentual / 100)

# TAUTOLÓGICO: reimplementa a fórmula
def test_aplicar_desconto():
    cupom = Cupom(percentual=Decimal("10"))
    total = Decimal("200")
    assert aplicar_desconto(total, cupom) == total - (total * cupom.percentual / 100)
```

TESTE TAUTOLÓGICO

Teste de comportamento

Valores independentes, limites e propriedade.

COMPORTAMENTO

```
class TestDesconto(unittest.TestCase):
    def test_desconto_percentual(self):
        casos = [("200.00", "10", "180.00"), ("0.00", "10", "0.00"), ("99.99", "100", "0.00"), ("10.00", "0",
"10.00")]
        for total, pct, esperado in casos:
            with self.subTest(total=total, pct=pct):
                self.assertEqual(aplicar_desconto(Decimal(total), Cupom(Decimal(pct))), Decimal(esperado))

    def test_cupom_acima_de_100_e_rejeitado(self):      # regra de negócio, não do código
        with self.assertRaises(ValueError):
            aplicar_desconto(Decimal("50.00"), Cupom(Decimal("101")))
```

TESTE TAUTOLÓGICO

A regra de negócio só você conhece

Cupom acima de 100% deve ser rejeitado: isso não está no código.

REGRA DE NEGÓCIO

```
def test_cupom_acima_de_100_e_rejeitado(self):      # regra de negócio, não do código
    with self.assertRaises(ValueError):
        aplicar_desconto(Decimal("50.00"), Cupom(Decimal("101")))
```

TESTE TAUTOLÓGICO

Teste que afirma o mock

Continua verde se a validação for apagada.

AFIRMA O MOCK

```
test("cria pedido", async () ⇒ {
  const repo = { save: jest.fn().mockResolvedValue({ id: 1 }) };
  const svc = new PedidoService(repo);
  const r = await svc.criar({ itens: [] });   // deveria rejeitar pedido sem itens
  expect(repo.save).toHaveBeenCalled();
  expect(r).toEqual({ id: 1 });                // é o que o próprio mock devolveu
});
```

Como detectar: sabotagem

1. Quebre o código sob teste de propósito: inverta um `if`, troque `<` por `⩽`, devolva constante
2. Se continuar verde, é suspeito
3. Pergunte: que erro real este teste pegaria?

COBERTURA E MUTAÇÃO

Cobertura mede o que foi executado, não o que foi verificado

Um teste sem asserção atinge 100% das linhas.

COBERTURA E MUTAÇÃO

Dois números para lembrar

4%

de escore de mutação em suítes com 100% de cobertura

Wang et al., MutGen · estudo com LLM em geração de testes

277

de 571 testes gerados seriam descartados se o critério fosse só cobertura

Meta ACH (Foster, Harman et al., 2025) · estudo com LLM em geração de testes

NOSSO CASO Cobertura de linhas de cerca de **90%** · escore de mutação de **74%**

Teste de mutação em uma frase

A ferramenta injeta falhas pequenas no código; mutante sobrevivente = a suíte não notou.

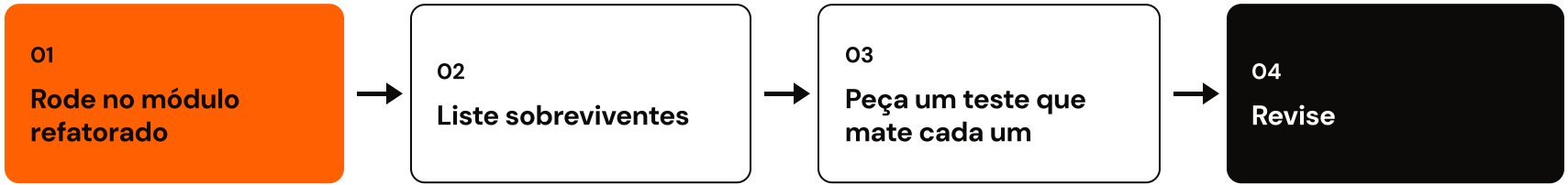


Ferramentas: `mutmut` · `Stryker` · `PIT`

```
$ python -B testes/mutacao_manual.py
```

Se interromper: `git status` e `git restore caso/miniloja`

Mutantes sobreviventes viram instrução para a IA



Postura adversarial: peça como o código falha

- Entradas vazias
- Nulas
- Limites
- Estouro
- Unicode
- Fuso horário
- Concorrência

ORÁCULO Você define o oráculo.

POSTURA ADVERSARIAL

Differential Prompting: a técnica real

1. O modelo infere a intenção
2. Sintetiza uma implementação alternativa
3. Compara por teste diferencial

Funciona melhor em programas simples.

28,8%

ChatGPT sozinho

QuixBugs

75,0%

com a técnica

QuixBugs

POSTURA ADVERSARIAL

Teste diferencial: legado x refatorado

Muitas entradas (grade determinística) contra o mesmo oráculo.

```
import itertools, unittest
from miniloja import calculo, calculo_legado      # calculo_legado = cópia congelada do original

CARRINHOS = [[], [{"preco": 1.0, "qtd": 2}], [{"preco": 100.0, "qtd": 2}], [{"preco": 250.0, "qtd": 1}]]

class TestDiferencial(unittest.TestCase):
    def test_refatorado_equivale_ao_legado(self):
        for itens, cupom, uf in itertools.product(CARRINHOS, [None, "PROMO10", "INVALIDO"], ["PE", "SP", "XX"]):
            with self.subTest(itens=itens, cupom=cupom, uf=uf):
                self.assertEqual(calculo.calcular_total(itens, cupom, uf), calculo_legado.calcular_total(itens,
cupom, uf))
```

DOCUMENTAÇÃO

Onde a IA ajuda e onde atrapalha em testes

AJUDA

Onde a IA ajuda

Funções puras, caracterização de legado, casos de borda, builders, bug virando teste.

ATRAPALHA

Onde a IA atrapalha

Infraestrutura real, concorrência, UI, autorização, regra que só está na cabeça do time.

DOCUMENTAÇÃO

Revisão de teste gerado

- [ ] A descrição diz o comportamento?
- [ ] O esperado é independente do código?
- [ ] Falharia com sabotagem?
- [ ] Há asserção observável?
- [ ] Há erro e limite?
- [ ] Dublê só nas fronteiras?

DOCUMENTAÇÃO

Documentação: parafrasear x registrar decisão

RUIM

```
# RUIM: parafraseia
def reservar(estoque, sku, qtd):
    """Reserva a quantidade do SKU no estoque."""
```

BOM

```
# BOM: contrato, motivo e armadilha
def reservar(estoque, sku, qtd):
    """Reserva `qtd` unidades para um pedido em checkout.
    Levanta EstoqueInsuficiente (sem reserva parcial). Idempotente por (pedido_id, sku).
    Expira em 15 min (regra do financeiro; ver ADR-0007). Não use para ajuste de inventário."""
```

DOCUMENTAÇÃO

O que a IA sabe e o que só você sabe

IA SABE

O que o código mostra

O que a função faz, tipos, fluxo.

VOCÊ FORNECE

O que só o time sabe

Por que existe, alternativas descartadas, restrições, quem depende.

Documentação que não apodrece

- [ ] Exemplos executáveis (doctest)
- [ ] Contratos de fonte única (OpenAPI)
- [ ] Checagem de links e drift em CI
- [ ] Docs no mesmo repositório

DOCUMENTAÇÃO

ADR com IA: quem faz o quê

VOCÊ

Decide

Contexto real, alternativas, decisão.

IA

Estrutura

Estrutura, lacunas, advogado do diabo.

AVISO **A IA não registra o que não foi decidido por pessoas.**

DOCUMENTAÇÃO

Exemplo de ADR

Usar Decimal para valores monetários

```
# ADR-0007: Usar Decimal para valores monetários
Status: Aceito · Decisores: time de Pedidos
## Contexto    Arredondamentos com float geraram divergência de centavos na conciliação.
## Decisão     Valores monetários passam a ser `Decimal`, com quantização explícita nas fronteiras.
## Consequências  + Conciliação reproduzível. - Migração de módulos; custo de serialização.
## Alternativas   Inteiros em centavos (descartado: conversão espalhada); manter float (descartado).
```

Janela: 15h10 às 16h30 Tempo encerrado

Caçar, mutar e derivar do contrato

(a) 3 testes suspeitos da suíte existente, com sabotagem

(b) 2 mutantes sobreviventes e o teste que os mataria

(c) 3 casos derivados de `caso/regras.md` (inclua a fronteira do frete grátis e um caso de arredondamento)

(d) ADR de 4 linhas, se o código discordar do contrato. **Não altere o esperado para o valor que o código devolve.**

Regras: modo manual · uma branch por tarefa · o agente não edita testes existentes · `/clear` entre as partes

Tempo: 80 min · **Roteiro:** `apoio/execucao-acompanhada-b3.md`

ONDE

`exercicios/aula-3/README.md`
`exercicios/aula-3/template-registro.md`

```
$ python -B testes/test_miniloja.py --codigo caso
--existentes
$ python -B testes/mutacao_manual.py
```

SEM AMBIENTE

`apoio/resultado-textual.md`

AV1.3: o que registrar

Testes suspeitos

Tipo, sabotagem, continuou verde?

Mutantes

Por que sobreviveu, teste que o mataria

Casos do contrato

Esperado calculado à mão, observado, decisão

ADR curto

4 linhas

NUNCA **altere o esperado para o valor que o código devolve**

VOLTEM ÀS

16h30

Tempo encerrado

Devolutiva individual

FECHAMENTO

Teste gerado que não falharia nunca não é teste, é enfeite

O esperado vem do contrato, não do código. Se vem do código, é espelho.

Sabotagem e mutação transformam "passou" em "verifica".

FECHAMENTO

Referências e estudo autônomo

Wang et al. (2023).
arXiv 2307.07221

Foster, Harman et al. (2025). Meta ACH.
arXiv 2501.12862

Wang et al. MutGen.
arXiv 2506.02954

Li et al. (2023). Differential Prompting.
arXiv 2304.11686

AV2

Até 21/10, 23h59

`avaliacoes/av2/README.md`

github.com/jamissonfreitas/repo_alunos_d12_esia

CESAR